Universidade Federal do Espírito Santo – Centro Tecnológico
Departamento de Informática
Prof. Bruno Légora

UFES

3ª Exercício Avaliativo (EA3)
Programação II (INF16153) - UFES
03 de fevereiro de 2025

Considere que você trabalha numa desenvolvedora de jogos 2-D "vistos de cima", como se você estivesse explorando um labirinto e possui visão de "cima para baixo". Para desenvolver tais jogos, sua empresa criou uma biblioteca de monitoramento de eventos para facilitar o tratamento dos mesmos. Sua biblioteca é composta por um TAD genérico que armazena um conjunto de eventos a serem monitorados e um TAD específico para cada evento. Para cada tipo de evento monitorado, sua biblioteca deve ter um TAD específico para tratá-lo. Desta forma, ao criar um novo tipo de evento, basta implementar seu novo TAD e utilizar o mesmo monitorador padrão. Seu monitorador deve ser capaz de registrar os eventos e processá-los quando eles ocorrerem. Sua biblioteca está nas etapas iniciais de desenvolvimento, e por enquanto ela deve suportar apenas 2 tipos de eventos: clique e movimento.

Evento movimento: ocorre quando um botão específico (Exemplo: W – cima, A – esquerda, S – baixo, D – direita) é apertado. O "jogador" deve se mover no ambiente, se possível, de acordo com a tecla apertada. Ex: Se ele está em (0,0), e a tecla apertada foi S (para baixo), ele deve se mover para a linha de baixo, ou seja, para a posição (1,0).

Evento clique: ocorre quando uma região da tela recebe um clique de mouse. O clique pode ser de 3 tipos: Botão esquerdo (L), botão do meio (M) e botão direito (R). Cada clique é representado pela letra do botão (L/M/R). Caso o clique seja L, um par de coordenadas é necessário, indicando que você quer selecionar um possível inimigo de uma posição adjacente. Caso seja M ou R, se o evento de clique possuir um inimigo selecionado que está do lado do jogador, este deve ser eliminado do mapa. Caso contrário uma mensagem deve informar que não há alvo, conforme casos de teste. Vale ressaltar que só é possível selecionar inimigos em coordenadas adjacentes (esquerda, cima, baixo e direita da posição atual).

Mensagens de erro são apresentadas em casos de movimento inválido, clique esquerdo que não selecionou inimigo, clique longe da posição atual, clique do meio/direito sem alvo selecionado. Olhe casos de teste ao final deste arquivo (e nos arquivos do script de correção).

Além disso, você deve implementar um TAD que representa o "ambiente de jogo", que vamos considerar ser um *grid* de tamanho e "configuração" fixos (o "mapa" do jogo é fixo), conforme fornecido nos arquivos deste EA. Para simplificar, considere que o *grid* é uma matriz de caracteres 10x10. O jogador inicia na posição (0,0) – no canto superior esquerdo do *grid* - e sua posição é modificada pelos eventos.

O monitorador de eventos faz cada evento interagir com o ambiente, que pode retornar a posição atual do jogador (e também alterá-la) e a "situação" de uma posição aleatória do mapa, conforme as seguintes possibilidades:

- 0 ou '\0', se for um espaço vazio (um espaço que o jogador pode se mover)

- '-' ou '|', se for uma parede horizontal ou vertical, respectivamente
- uma letra qualquer, representando o identificador de um inimigo

Em caso de "sucesso" do evento (ex: movimentar para um espaço vazio), o evento deve alterar o ambiente usando a interface do mesmo. O ambiente aceita ser alterado de duas formas possíveis: alterar posição do jogador e alterar conteúdo da posição.

Exemplo: O jogador está na posição (2,3). Um clique na posição (3,3), que é adjacente à posição (2,3) retorna "\0" se e somente se (s.s.s.) a posição (3,3) estiver vazia.
Exemplo 2: Um movimento ocorre s.s.s. o espaço adjacente for vazio na direção do movimento. Então, o ambiente deve ser "alterado" para que o jogador passe a estar na nova posição.
Exemplo 3: O sistema registra um ataque ao inimigo selecionado anteriormente (por clique com o botão esquerdo – L) s.s.s. ele estiver em uma posição adjacente. Para facilitar, vamos considerar que todo ataque elimina um inimigo. Dois ataques são suportados, quando utilizados através dos cliques "do meio" e "da direita" (caracteres M ou R, respectivamente).

Todos os eventos são recebidos via entrada padrão (teclado) até que seja recebido um evento inválido (ou seja, uma linha com apenas um caractere não registrado no jogo).

Regras de funcionamento:

- Sua main deve ter o seguinte comportamento:
  - Ler o modo de jogo (ver próximo tópico)
  - Inicializar as variáveis necessárias para o funcionamento do jogo
  - Realizar o "monitoramento" dos eventos que alteram o jogo, imprimindo as mensagens conforme exemplos
- Três modos de jogo são suportados, e devem ser lidos inicialmente na main:
  - Modo 1: Clique + movimento W A S D
  - Modo 2: Clique + movimento U H J K
  - Modo 3: Clique + movimento W A S D + movimento U H J K (o movimento pode ser feito de duas maneiras distintas em qualquer ordem)
  - W/U movimentam "para cima" (diminui 1 da linha da matriz); A/H movimentam "para a esquerda" (diminui 1 da coluna da matriz), S/J movimentam "para baixo" e D/K para "a direita".
  - Cuidado! Digitar "U" no modo 1 significa finalizar o jogo, conforme parágrafo anterior.
- O monitoramento de eventos consiste em ler dados do teclado e, a medida com que teclas vão sendo digitadas, os respectivos eventos associados a elas (se existirem) vão sendo processados/executados. Caso não tenha evento associado à tecla apertada, o programa deve ser encerrado

Observações/Dicas/Restrições:

- O mapa é fornecido no arquivo mapa.txt enviado junto com esta descrição. Seu TAD ambiente deve armazenar a matriz e registrar todas as possíveis mudanças.
- Todos os scanf do gabarito foram implementados sem \n no final, mas seguidos de um getchar(). Por exemplo, para ler o modo de jogo, foi feita a leitura com "%d", e na linha abaixo um getchar(). Recomendo seguirem esse padrão em todos os scanf para evitarem problemas.
- As coordenadas são sempre inteiras e maiores ou iguais a zero!
- A posição (x,y) é adjacente a posicao (x_atual,y_atual) se a eq. abaixo for igual a 1:

$$(x_atual - x)^2 + (y_atual - y)^2$$

- **Propositalmente, o TAD do evento de clique não foi fornecido nos arquivos deste EA**. O TAD monitorador é genérico e deve suportar eventos de diferentes tipos, incluindo aqueles que ainda não foram previstos. Você deve criar o .h e o .c do evento Clique, seguindo os padrões combinados em sala de aula (TAD opaco).
  - Ele é avaliado apenas no teste "main" do script de correção. Neste teste, você deve fazer tanto a main quanto o evento clique
  - O evento clique deve aceitar apenas as 3 teclas: L, para botão esquerdo, M para botão do meio, e R para botão direito.
- Não há um objetivo no jogo, por enquanto. O desenvolvimento do jogo está em uma etapa apenas de exploração do mapa e eliminação instantânea de inimigos, até receber um evento não previsto.
- Olhe os casos de teste para saber quais mensagens você deve imprimir! No fim deste arquivo, são apresentados alguns exemplos.
- O código deve seguir as interfaces definidas nos arquivos ".h" fornecidos junto com o exercício. Esses arquivos **não devem ser modificados**, pois contêm as especificações do problema a ser resolvido.
- Você será responsável pela manipulação correta da memória durante a execução do programa. Isso inclui a alocação dinâmica e a liberação de memória conforme necessário, garantindo que não haja vazamentos de memória.
- Você está recebendo alguns casos de teste. Há casos de teste ocultos que não serão executados no BOCA. Dessa forma, a nota que o script retornar no seu computador pode ser alterada devido a esse novo caso de teste.
  - Há um tipo de evento "escondido" que testará se o TAD genérico de vocês realmente foi genérico.
- **Não esqueçam de incluir as bibliotecas que vocês usarem.**

<u>Padrão de entrada e saída</u>

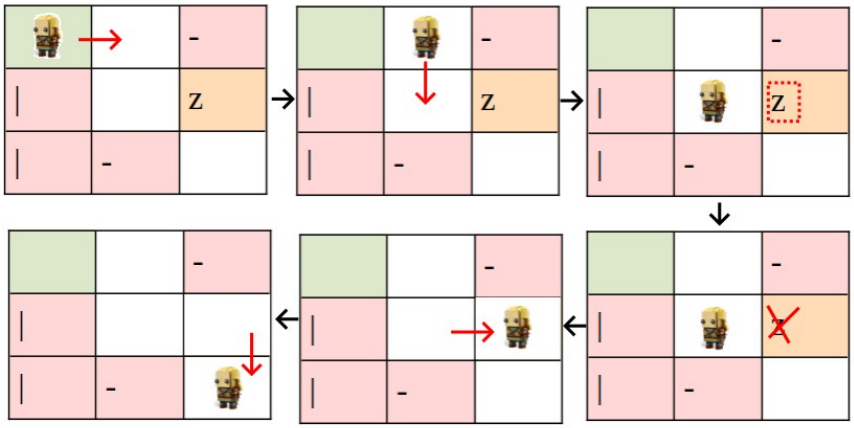
Seu programa deve iniciar lendo o modo de jogo do teclado. As próximas leituras serão as dos possíveis eventos que modificam o mapa. Neste documento, um exemplo minimalista (mapa 3x3) é apresentado:

		-
|		z
|	-	

No exemplo acima, o mapa possui 4 posições “livres” – sem nenhum obstáculo ou inimigo, 4 posições com obstáculos fixos (paredes, representadas em vermelho), e uma posição com o inimigo z (que pode ser removido com um ataque). O jogo inicia na posição (0,0), em verde. No exemplo abaixo, vemos um exemplo de entrada para este mapa:

- Modo 1 (clique + movimento WASD)

```
1  // Modo de jogo 1
W // Tenta mover para cima, mas não pode (está no topo)
A // Tenta mover para esquerda, mas não pode (está na lateral esquerda)
S // Tenta mover para baixo, mas não pode (há obstáculo)
D // Move para direita com sucesso
L // Clique botão esquerdo
1 2 // Posição de clique. Dará erro pois está longe
S // Move para esquerda com sucesso
D // Tenta mover para direita, mas não pode (há inimigo impedindo)
M // Clique botão do meio. Erro pois não há alvo selecionado
L // Botão clique esquerdo
1 2 // Posição de clique. Dará certo pois está do lado: na pos (1,1)
M // Botão do meio – Elimina o inimigo selecionado
D // Move para a direita, pois está livre
S // Move para baixo.
F // Evento não registrado, finaliza programa.
```

- Saída:

```
Modo 1 selecionado!
MOVIMENTO INVALIDO!
MOVIMENTO INVALIDO!
MOVIMENTO INVALIDO!
MOVIMENTO PARA POSICAO (0,1)
CLIQUE LONGE DA POSICAO ATUAL
MOVIMENTO PARA POSICAO (1,1)
MOVIMENTO INVALIDO!
CLIQUE DO MEIO SEM ALVO
INIMIGO z SELECIONADO EM (1,2)
INIMIGO z DA POSICAO (1,2) ELIMINADO COM CLIQUE DO MEIO
MOVIMENTO PARA POSICAO (1,2)
MOVIMENTO PARA POSICAO (2,2)
```